

@monitor/sdk-tracer — 다른 SDK 사용자를 위한 계측 라이브러리

Agent Tracer는 현재 **Claude Code** 플러그인과 **Codex hook** 어댑터만 자동 계측한다. Anthropic `@anthropic-ai/claude-agent-sdk` 로 직접 만든 사내 봇·평가 하니스·고객 대상 AI 기능은 수동 HTTP 호출로만 합류할 수 있다. **@monitor/sdk-tracer**는 SDK의 `query()`를 얇게 감싸서 같은 privacy contract · 같은 ingest 페이로드 · 같은 대시보드 surface를 **한 줄 교체**로 가져가게 한다.

새 패키지 `packages/sdk-tracer/` 의존 `@anthropic-ai/claude-agent-sdk` 대상 ingest POST `/ingest/v1/*`
npm 공개 후보 `@monitor/sdk-tracer`

0 TL;DR

PUBLIC API

2

`tracedQuery, tracedSession`

설치 후 변경

1 line

`import { tracedQuery }`

대시보드 변경

0

기존 surface 그대로 재사용

MVP까지

~5d

Phase 1-4

1 풀고 싶은 문제

Claude Code의 혹은 “사용자가 IDE에서 직접 쓰는 인터랙티브 세션”에만 붙는다. 그런데 실제 조직에서는 SDK를 통해 **혹이 닿지 않는 환경**에서도 Claude가 돌아간다 — 슬랙 봇, 야간 batch, CI 안의 코드 리뷰어, 고객 채팅에 임베드된 에이전트. 이들에 대해서도 “정확히 어떤 동작을 했나” “룰을 지켰나”를 같은 도구로 보고 싶다.

이 격차는 Agent Tracer의 가장 큰 채택 장벽이기도 하다. **Claude Code 한정 도구**로 보이면 SDK 기반 프로덕션 워크로드를 운영하는 팀은 다른 솔루션을 찾는다.

2 유즈케이스

각각의 케이스는 모두 “SDK 환경에서 Agent Tracer를 그대로 쓰고 싶다”를 다른 옷으로 입은 것.

UC-A Slack 봇 — 사내 PM이 “요청 사항을 PR로 만들어줘”

Long-lived process

Express + Slack Bolt 위에 `query()`로 짠 봇. 동일 channel-thread를 같은 session으로 묶어 멀티-턴 대화 유지. 운영 중 “왜 봇이 user repo의 잘못된 파일을 만졌지?”가 자주 발생.

현재

로그를 grep으로 뒤집. `tool_use`가 정형화돼 있지 않아 어떤 파일을 만졌는지 한눈에 모름.

TRACEDQUERY 적용 후

슬랙 thread = task. 대시보드에 자동 등록. 룰 “.env 수정 금지”가 Claude Code 세션과 동일하게 작동.

UC-B 평가(eval) 하니스 — 한 프롬프트를 1,000번 돌려 모델 비교

Batch

ML 팀이 nightly로 “refactor 100개 후보 prompt”를 `claude-haiku-4-5`와 `claude-sonnet-4-6`로 각각 돌린다. 한 모델이 다른 모델보다 “덜 파일을 만진” 케이스를 찾고 싶음.

현재

결과 JSON을 별도 노트북에서 집계. 룰 위반/통과는 수동 분석.

TRACEDQUERY 적용 후

매 평가 = 1 task. 같은 룰 세트 + Judge Loop가 자동 평가 → 모델 간 **verdict** 분포 비교가 대시보드 차트.

UC-C CI 안의 PR 리뷰어 — GitHub Action 컨테이너

Ephemeral

PR 열릴 때 컨테이너 안에서 `query()`로 코드 리뷰 코멘트를 생성. 컨테이너는 분 단위로 사라지고, 로그는 휘발됨.

현재

PR 코멘트만 남고 “리뷰어가 어떤 파일을 살펴봤는지”는 사라진다. 룰 적용 불가.

TRACEDQUERY 적용 후

컨테이너에서 `MONITOR_BASE_URL`로 사내 모니터에 push. PR ID = task ID. 사후 “이 리뷰어가 PR 외 파일을 봤나?”를 검증.

사용자 “내 데이터 분석 좀”에 응답하는 인앱 어시스턴트. 다수의 동시 세션, 각 세션은 분~시간 단위.

현재

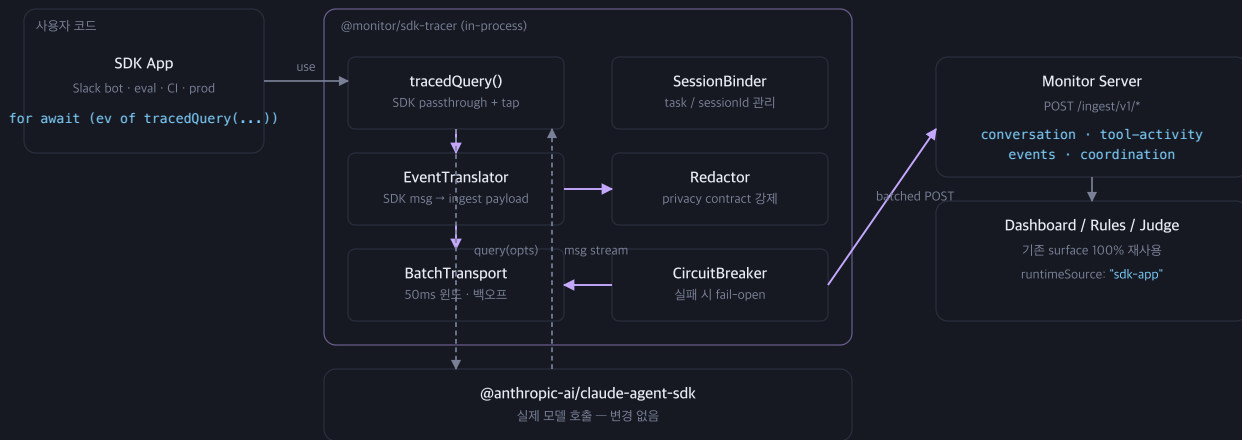
에러율과 latency만 APM에 있음. “어떤 query를 어떤 순서로 실행했는지”는 부분 노출.

TRACEDQUERY 적용 후

user_id → task tag로 매핑. 룰: “고객 PII 컬럼을 평문 출력 금지”. 위반 시 PreToolUse 차단 로직과 동일한 의미의 사후 알람.

3 아키텍처

핵심 아이디어: 사용자는 평소처럼 SDK 메시지 스트림을 받는다. **tracer**는 같은 스트림을 옆에서 한 번 더 듣고 이벤트 변환기로 ingest payload를 만들어 모니터에 비동기 POST. 사용자 코드 입장에선 SDK가 똑같이 동작하고, 네트워크 실패 시에도 메인 에이전트는 절대 막히지 않는다.



① tracer는 SDK 모듈 자체를 패치하지 않는다. 래퍼일 뿐 — 사용자가 평소처럼 query()도 같이 쓸 수 있음

그림 1. 사용자 코드는 SDK 메시지 스트림을 받는다(투명 전달). tracer는 옆에서 같은 스트림을 듣고 변환·전송한다.

4 이벤트 변환 표 — SDK 메시지 → ingest payload

이 표가 핵심 산출물. SDK가 내는 모든 메시지 종류에 대해 “무엇을 보내고” “무엇을 버리는지”를 정의한다. Codex 어댑터의 action-only 정책과 동일.

SDK 메시지	매핑되는 INGEST	전송하는 필드 (KEPT)	버리는 필드 (DROPPED)
system.init	POST /ingest/v1/sessions/ensure + tasks/start	session_id, model, cwd, tools_available[]	—
user (text)	POST /ingest/v1/conversation {kind:"user.message"}	prompt 텍스트 (raw 캡처 ok)	—
user (tool_result)	— (서버 저장 안 함)	tool_use_id (상관관 계용)	result.content 전체
assistant (text)	POST /ingest/v1/conversation {kind:"assistant.response"}	message text	—
assistant (thinking)	events {kind:"context.saved", lane:"planning"}	thinking 요약 (올 션)	signature 등 내 부 메타
tool_use (Bash)	tool-activity {kind:"terminal.command"}	command, description, timeout	env, stdin
tool_use (Read/Edit/Write)	tool-activity {kind:"tool.used"}	file_path, offset/limit, edit count	old_string, new_string 본문
tool_use (Grep/Glob)	tool-activity (exploration lane)	pattern, path, glob, output_mode	matched line content
tool_use (WebFetch/WebSearch)	tool-activity (exploration lane)	url(s), query, allowed_domains	response body 전체
tool_use (mcp__*)	coordination	tool name, input keys	민감 인자 (자동 마 스킹)
tool_use (Agent)	coordination + tasks/link	subagent_type, description	subagent prompt 본문
result	sessions/end + telemetry	stop_reason, usage(token), latency, cost	—

① “kept”는 기존 Claude Code 혹은 캡처하는 것과 같은 집합. 즉 SDK 경로에서도 같은 데이터 모양이 나오므로 Inspector / Rules / Judge 모두 별도 분기 없이 동작.

5 Public API 디자인

두 가지 진입점. `tracedQuery`는 1회용·짧은 호출에, `tracedSession`은 대화형·장기 세션에.

```
// packages/sdk-tracer/src/index.ts
export interface TracedQueryOptions extends SDKQueryOptions {
  task?: {
    id?: string;
    title?: string;           // 기본: prompt 첫 80자
    source?: string;          // 기본: "sdk-app"
    tags?: Record<string, string>;
    parentTaskId?: string;
  };
  monitor?: {
    baseUrl?: string;         // 기본: env MONITOR_BASE_URL → http://127.0.0.1:3847
    apiKey?: string;
    failureMode?: "silent" | "throw"; // 기본 silent
    flushOnExit?: boolean;    // 기본 true
  };
  ruleSync?: {
    inject?: boolean;         // 활성 룰을 system prompt로 주입
    pollIntervalMs?: number;
  };
}

export async function* tracedQuery(
  opts: TracedQueryOptions
): AsyncGenerator<SDKMessage> { /* yield 사용자에게, side-channel 전송 */ }

export class TracedSession {
  send(prompt: string): AsyncGenerator<SDKMessage>;
  checkpoint(label: string): Promise<void>;
  end(): Promise<void>;
  readonly taskId: string;
  readonly sessionId: string;
}

export function tracedSession(opts: TracedQueryOptions): TracedSession;
```

6 시퀀스 — `tracedQuery` 한 번의 흐름

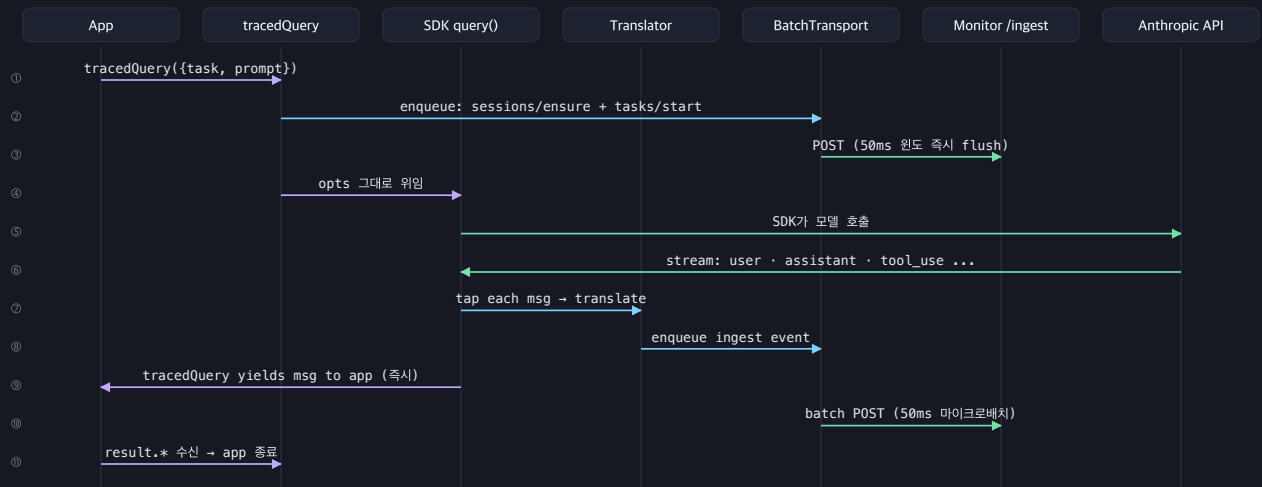


그림 2. ㉔가 ㉔㉔과 분리된 게 포인트 — 메시지는 먼저 **app**에 **yield**되고, 전송은 별도 마이크로배치 큐로 들어간다.

7 장애 격리 — 모니터가 죽어도 에이전트는 산다

Production 채택의 마지노선은 “계측이 본체를 죽이지 않을 것”. **fail-open + circuit breaker + bounded queue** 3겹.

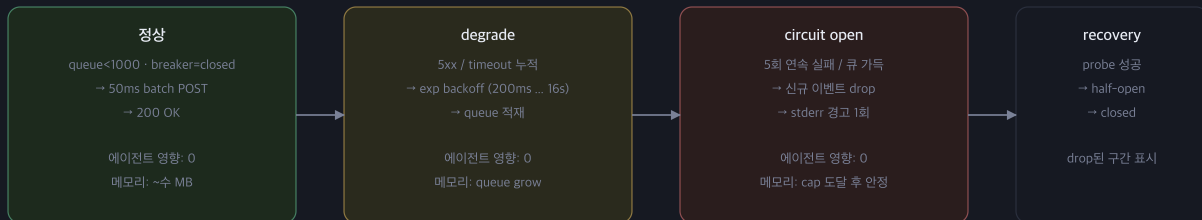


그림 3. 4-상태 머신. 모든 상태에서 “사용자 SDK 호출의 latency 추가 = 0” 보장.

8 도입 — 4 스텝

STEP 1

설치

모니터 서버는 별도 실행(이미 있다면 skip).

STEP 2

환경 변수

도커 / k8s ENV에 한 줄.

```
npm i @monitor/sdk-tracer
@anthropic-ai/claude-agent-sdk
```

```
MONITOR_BASE_URL=
http://monitor.internal:3847
```

STEP 3

임포트 교체

모든 호출 사이트에서 한 줄.

```
- import { query } from
- "@anthropic-ai/claude-agent-sdk"
+ import { tracedQuery as query }
+ from "@monitor/sdk-tracer"
```

STEP 4

task 메타데이터

대시보드에서 보고 싶은 묶음 단위를 옵션으로.

```
tracedQuery({
  prompt,
  task: { title, source:
    "slack-bot", tags: { ch }}
})
```

9 패키지 레이아웃

```
packages/sdk-tracer/ └─ package.json // peer: @anthropic-ai/claude-agent-sdk └─
README.md └─ src/ │ └─ index.ts // public API │ └─ adapter/ │ │ └─ event-translator.ts
// SDK msg → ingest event (4장 표) │ │ └─ redactor.ts // privacy contract 강제 │ │ └─
session-binder.ts // task / sessionId 관리 │ │ └─ rule-sync.ts // 옵션: 활성 룰을 system
prompt에 inject │ └─ transport/ │ │ └─ batch-transport.ts // 50ms 윈도우 micro-batch │ │
└─ circuit-breaker.ts │ │ └─ bounded-queue.ts // 메모리 cap │ └─ types/ │ └─ sdk-
messages.ts // SDK 타입 재export │ └─ ingest-payloads.ts // 서버 스키마와 1:1 │ └─ test/ │ └─
translator.spec.ts // 모든 SDK 메시지 종류별 │ └─ redactor.spec.ts // privacy 누락 회귀 방지 │
└─ resilience.spec.ts // failure isolation └─ examples/ └─ slack-bot/ └─ eval-batch/
└─ ci-reviewer/
```

10 구현 계획 (Phase 단위)

총 10~12 사람-일. Phase 1-4가 "한 줄 교체로 동작"까지 도달. Phase 5-7은 채택 후 가치를 키우는 옵션.

P0

패키지 스캐폴딩

workspace 추가, tsup build, peer dep 설정, ingest 페이로드 타입을 서버에서 import 하도록 경로 별칭.

1 day

P1

EventTranslator

4장 표의 12행을 함수로. 단위 테스트 — 각 SDK 메시지 fixture → 기대 ingest payload.

2 days

P2

Redactor + privacy 회귀 테스트

tool_use 입력 화이트리스트, tool_result 통째 drop, snapshot 테스트로 “새 필드가 새어나오면 빨강게”.

1 day

P3

BatchTransport + CircuitBreaker

50ms 윈도 micro-batch, 지수 backoff, bounded queue, 종료 시 flush. resilience 테스트(모니터 down/지연/timeout).

1 day

P4

tracedQuery + tracedSession 공개 API

async generator passthrough, sessionId 관리, exit hook으로 SIGTERM 처리.

1 day

P5

예제 3종

Slack bot, eval batch, CI reviewer. README의 “이거 보고 따라하면 끝” 진입점.

2 days

P6

RuleSync — 활성 룰을 system prompt에 주입

모니터의 /api/v1/rules?source=human&active=true를 주기적 polling. NL 룰 텍스트를 system 블록으로 prepend. 에이전트가 룰을 인지한 채 동작.

2 days · opt

P7

OTEL bridge

@opentelemetry/api가 process에 있으면 같은 span을 OTEL로도 발행. 기존 APM과 양립.

2 days · opt

P8

npm 공개 + 버전 정책

0.x → 1.0 게이트는 "4장 표 변경이 90일간 0건". SDK 버전 매트릭스 README에 명시.

1 day · opt

11 사용 예제

UC-A · Slack 봇

```
import { App } from "@slack/bolt";
import { tracedSession } from "@monitor/sdk-tracer";

const bolt = new App({ token: env.SLACK_TOKEN, signingSecret: env.SLACK_SECRET });
const sessions = new Map<string, ReturnType<typeof tracedSession>>();

bolt.message(async ({ message, say }) => {
  const key = message.thread_ts ?? message.ts;
  let s = sessions.get(key);
  if (!s) {
    s = tracedSession({
      task: { title: message.text.slice(0, 80),
        source: "slack-bot",
        tags: { channel: message.channel, user: message.user } },
      monitor: { failureMode: "silent" }
    });
    sessions.set(key, s);
  }
  for await (const ev of s.send(message.text)) {
    if (ev.type === "assistant" && ev.text) await say({ text: ev.text, thread_ts: key });
  }
});
```

UC-B · 모델 비교 eval batch

```
import { tracedQuery } from "@monitor/sdk-tracer";
import { prompts } from "./fixtures.js";

for (const model of ["claude-haiku-4-5-20251001", "claude-sonnet-4-6"]) {
  for (const p of prompts) {
    for await (const _ of tracedQuery({
      model, prompt: p.text,
      task: { title: `eval ${model} #${p.id}`, source: "eval-batch",
```

```

        tags: { model, suite: p.suite } }
    ))) {} // stream 소비만 - 결과는 대시보드에서 확인
}
}
// 끝나면 Dashboard에서 source=eval-batch, tags.model로 grouping → verdict 분포 비교

```

UC-D · 실패 격리 (모니터가 죽어도 진행)

```

for await (const ev of tracedQuery({
  prompt, task: { title, source: "prod-agent", tags: { user_id } },
  monitor: {
    baseUrl: env.MONITOR_BASE_URL,
    failureMode: "silent", // 절대 throw 하지 않음
    flushOnExit: true
  }
})) {
  // 모니터 down 이어도 ev는 그대로 흐른다. circuit open 시 stderr에 1회만 경고.
  handleUser(ev);
}

```

12 리스크와 완화

SDK 메시지 스키마 변화

Anthropic이 SDK 메시지 모양을 바꾸면 translator가 깨짐. 완화: SDK 버전 매트릭스 + contract 테스트(실제 fixture 캡처). semver-major 표시.

privacy 누락 회귀

tool_use 새 종류 추가될 때 화이트리스트 미반영. 완화: snapshot 테스트로 "새 필드 등장 시 자동 빨강". 기본 동작은 "모르면 drop".

SDK 호환성

SDK 메이저 업그레이드 시 사용자가 둘을 동시에 올려야 함. 완화: peerDep으로 명시, 범위 좁힘, examples를 CI에서 실제 SDK 버전으로 빌드.

대량 trace로 모니터 과부하

UC-B처럼 1000건 동시 발생. 완화: 서버 측 ingest는 이미 batch + idle backoff(perf/queue 커밋). tracer는 50ms micro-batch로 1차 압축.

인증 / 멀티 테넌시

모니터는 현재 로컬 신뢰 모델. 사내/공용 배포 시 API key 필요. 완화: `monitor.apiKey` 옵션을 동시에 도입하고 서버 측 미들웨어 추가(별도 작업이지만 인터페이스는 미리).

OpenInference / OTEL 이중 export

OTEL bridge 활성화 시 span 중복. 완화: tracer의 OTEL bridge는 opt-in이고, 서버 측 OpenInference export와 동일 `trace_id` 규약 사용해서 중복 검출 가능.